CRAFT: COMPOSITIONAL REWARD ALIGNMENT FOR TARGET-HIDDEN LOOP VERIFICATION

Anonymous authors
Paper under double-blind review

ABSTRACT

Valid loop invariants are closed under conjunction: useful clauses from different responses can prove a target even when no response does so alone. Yet standard training scores each response as an indivisible answer. We study this mismatch under *target-hidden loop verification*, where target assertions and postconditions Q are withheld during generation and training. CRAFT aligns both stages through rollout–decompose–filter–compose. At inference, it decomposes responses into clauses, filters them with syntax checking and Houdini, and composes an inductive subset before restoring Q. SFT distills a filtered multi-response union into one response. RL assigns credit within a rollout to its verifier-retained subset and across rollouts to complementary coverage beyond peers. Its target-independent strength signal comes from reachable executions and conservatively filtered synthetic negatives. On 832 C programs, four `gpt-5-nano` rollouts verify 49.0% of tasks, versus 42.2% for the strongest external baseline under each system's fixed budget.

1 INTRODUCTION

Loop invariants turn unbounded executions into finite proofs, but a formally valid invariant can still be useless. An invariant is *valid* if it holds when execution first reaches the loop and remains true after every loop iteration. A verifier can check these obligations, yet its pass/fail outcome provides no graded measure of how much the invariant says: $true$ is valid for every loop but proves nothing about its behavior or desired outcome. A useful invariant must also be strong enough to establish downstream properties. Since the exact reachable-state set may be expensive to compute or inexpressible in the annotation language, generation must navigate many valid but weak over-approximations and receive credit for approaching stronger ones.

This search need not succeed in a single attempt. Validity is closed under conjunction: the conjunction of two valid invariants is valid and no weaker than either one. Useful clauses can therefore be accumulated across samples instead of demanding a complete answer from one response. Model responses expose such partial results as clauses: one may discover a range bound, another a relational equality, and together they may prove a target that neither proves alone. Clause-wise verification can discard invalid clauses without losing the rest. The natural unit of inference is therefore the filtered clause pool, not a complete response.

Response-level rewards are mismatched to this unit in two ways. First, verifiable rewards credited to whole responses chiefly sharpen the sampling distribution toward responses the policy already produces, improving pass@1 without widening what k samples can cover (Yue et al., 2025). For a clause pool, such sharpening toward a few individually complete responses is at best orthogonal and at worst collapses the diversity that composition exploits. Second, a complete response is also the wrong unit of training credit. A whole-response reward withholds credit from useful clauses whenever another clause in the same response fails. A reward computed independently for each response cannot distinguish a constraint that expands group coverage from one already repeated by every peer. Under repeated sampling, this can encourage the policy to reproduce easy high-reward clauses instead of exploring a clause pool whose members work together. Our central claim is that loop-invariant learning should optimize the verified composition obtained from a fixed rollout group, rather than treating every rollout as an independent final answer.

We study this claim under *target-hidden loop verification*. The model sees the function preconditions, executable prefix, loop guard, and body, but the target set Q is withheld until the generated invariant

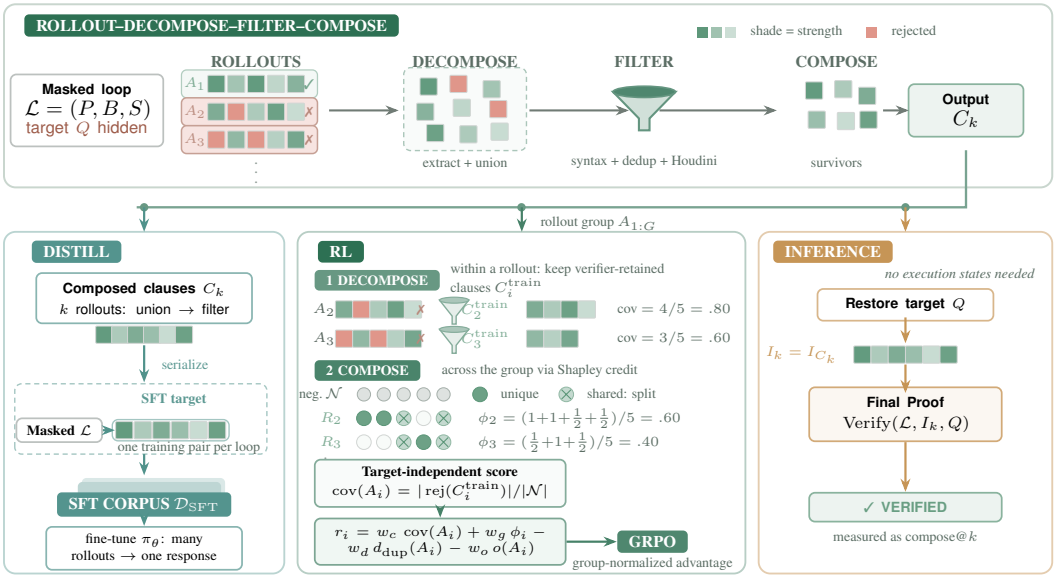

Figure 1: **Rollout–decompose–filter–compose is the organizing principle.** Target-hidden rollouts are parsed, unioned, and Houdini-filtered. SFT distills the composed set. RL *decomposes* each rollout onto its verifier-retained clauses—a dense, target-independent strength score from execution-derived negatives—and *composes* credit across the rollout group through a closed-form Shapley allocation, so a rejection already supplied by peers earns less. Executions construct and clean only training-time reward signals; inference restores Q only for the final proof.

set has been fixed. The final test still uses the original task: after Q is restored, the verifier must establish the loop obligations and prove every target at its original program point. Hiding Q prevents a model from copying a target that happens to be valid or using target-specific proof failures to steer generation. It also rules out target success as a training reward, making a graded, target-independent measure of invariant strength necessary.

CRAFT organizes inference around *rollout–decompose–filter–compose*. Given a budget of k target-hidden responses, the pipeline extracts each invariant annotation as a clause, unions clauses across responses, removes syntax failures, and applies Houdini to retain a jointly valid subset. Only then is Q restored for the final proof. We measure this inference-time object with compose@k: the fraction of programs verified after composing the first k responses. Unlike pass@k, which requires at least one response to contain the complete proof, compose@k preserves useful partial answers. The objective is to make this curve both higher and faster to converge, so a small rollout budget recovers the compositions otherwise found only through much broader sampling.

Training aligns credit with the two levels of this inference procedure. *Within* each rollout, we project its extracted clauses onto a verifier-retained subset; one rejected clause does not erase credit earned by the others. We score the strength of that subset using the execution-derived negative examples it rejects, without adding a reward merely for well-formed syntax or whole-response validity. *Across* rollouts, we apply a closed-form Shapley allocation to the union of their independently retained negative-example coverage: repeated rejections share credit, whereas a rejection supplied by fewer peers receives more. Thus the reward of one rollout depends on what the other rollouts in its Group Relative Policy Optimization (GRPO) group discover (Shao et al., 2024). Synthetic supervised fine-tuning (SFT) additionally distills a filtered multi-response union into one individual response. Together, these stages make sampled responses more useful to the clause pool that the pipeline composes at inference.

Prior systems synthesize and prune candidate invariants at inference time, while learning-based methods train from verified outputs, solver feedback, or behavioral examples (Flanagan & Leino, 2001; Kamath et al., 2023; Cao et al., 2025; Si et al., 2020; Yan et al., 2025; Yang et al., 2026a; Huang et al., 2026). We instead train for the filtered clause pool used at inference: credit preserves useful clauses within a response and depends on complementary coverage across responses.

Our experiments deliberately separate standalone success from composed inference. On 832 integer C programs, four target-hidden `gpt-5-nano` responses with union and Houdini verify 408 programs (49.0%), compared with 351 (42.2%) for the strongest external baseline under each system's fixed budget.

Contributions. This paper makes three contributions:

- We formulate target-hidden loop verification around a fixed-budget rollout–decompose–filter–compose objective. The evaluated object at inference is the verified composition measured by compose@k, not an isolated response.
- We align learning credit with that object in both directions: the reward preserves useful clauses within imperfect rollouts and allocates complementary coverage across rollouts, without using Q.
- We show that this alignment improves composed verification and rollout efficiency, and audit the execution-derived examples that provide its graded strength signal.

2 PRELIMINARIES

Loop verification. A task consists of a loop $\mathcal{L} = (P, B, S)$ and a target set $Q = \{(\ell, q_\ell)\}$, where predicate q_ℓ occurs at assertion or postcondition location ℓ. Here P describes the loop-entry states induced by the function preconditions and executable prefix, B is the guard, and S is the body. Let $\mathrm{Reach}(\mathcal{L})$ be the reachable loop-head states, including the final guard test. An invariant I is *valid* (or inductive) when

$$P \Rightarrow I, \qquad \{I \wedge B\}\, S\, \{I\},$$

which ensures $\mathrm{Reach}(\mathcal{L}) \subseteq [\![I]\!]$ (Floyd, 1967; Hoare, 1969). The semantic ideal is an exact invariant $I^\star$ with $[\![I^\star]\!] = \mathrm{Reach}(\mathcal{L})$, although this set need not be finitely expressible in the annotation language.

For predicates I and J, write

$$I \preceq J \quad \Longleftrightarrow \quad I \Rightarrow J \quad \Longleftrightarrow \quad [\![I]\!] \subseteq [\![J]\!]\,.$$

Thus $I \preceq J$ means that I is at least as strong as J, and $I \prec J$ means that it is strictly stronger. For a finite clause set C, define

$$I_C \triangleq \bigwedge_{c \in C} c, \qquad I_\varnothing \triangleq \mathit{true}.$$

We call C *jointly inductive* when I_C is valid, equivalently when

$$P \Rightarrow I_C, \qquad \{I_C \wedge B\}\, S\, \{I_C\}.$$

We reserve $\mathsf{H}_{\mathcal{L}}(C)$ for the ideal Houdini fixed point under exact establishment and preservation decisions, and $\mathsf{Filter}_{\mathcal{L}}(C)$ for the implemented parsing, deduplication, and resource-bounded WP/Houdini pipeline.

A valid invariant is Q-*sufficient* when annotating the loop with I lets the verifier prove every q_ℓ at ℓ. For one post-loop target this reduces to $I \wedge \neg B \Rightarrow q$. Thus validity is target-independent, whereas sufficiency is tested only after restoring Q.

Target-hidden verification. The generator and all intermediate scores use $\mathcal{L} = (P, B, S)$, not Q. Assertions, postconditions, executable assertion calls, and conventional error-label names are masked; non-contract comments, including source-provenance paths, are removed. Preconditions and executable loop semantics remain visible. The generated invariant is fixed before the untouched program is restored for final verification. We study scalar-integer C programs with one loop; Appendix A gives the scope and a motivating nonlinear example.

3 RELATED WORK

Candidate generation, filtering, and prompt-time composition. Classical invariant synthesis uses abstract interpretation, affine relations, templates, interpolation, recurrences, syntax-guided search, or counterexamples; execution-driven systems instead mine likely properties from traces (Cousot

& Cousot, 1977; Karr, 1976; Colón et al., 2003; McMillan, 2003; Kincaid et al., 2017; Alur et al., 2013; Garg et al., 2014; Ernst et al., 2007). Houdini iteratively removes invalid candidates to retain an inductive subset (Flanagan & Leino, 2001). LLM workflows reuse this pattern: Loopy couples proposals with Houdini and repair, iRank ranks candidates, and Clause2Inv composes retained clauses while iteratively accumulating counterexamples (Kamath et al., 2023; Chakraborty et al., 2023; Cao et al., 2025). LaM4Inv, ACInv, AutoSpec, SpecGen, SESpec, LEMUR, and Baldur similarly organize generation with static analysis, symbolic execution, verification, or repair (Wu et al., 2024a; Liu et al., 2024b; Wen et al., 2024; Ma et al., 2025; Yang et al., 2026b; Wu et al., 2024b; First et al., 2023). These are inference workflows; CRAFT fixes all target-hidden rollouts before joint filtering, receives no verifier feedback between calls, and trains responses for fixed-group composition.

Specialized neural invariant learning. Code2Inv learns a construction policy from solver feedback; CLN2INV and G-CLN learn continuous representations of linear and nonlinear relations; and LIPuS composes RL-based pruning with SMT solving (Si et al., 2018; 2020; Ryan et al., 2020; Yao et al., 2020; Yu et al., 2023). These task-specific models optimize one candidate or construction trajectory. We instead post-train a general language model so independently sampled responses contribute jointly under a fixed budget.

Behavioral-example objectives for specification strength. COINS assesses specifications using trusted positive and mutation-derived negative input–output cases (Yang et al., 2026a); SpecRL rewards complete method-level specifications that reject impossible input–output pairs (Huang et al., 2026); it neither targets loop invariants nor derives negatives from loop dynamics. Both show that behavioral examples distinguish weak but valid specifications from stronger ones. CRAFT supplies that loop-specific construction, filters each imperfect response to an inductive clause subset, and conditions its credit on negative states already rejected by peer rollouts.

Language-model training from formal feedback. InvBench and Wonda use verified data for invariant fine-tuning; CodeRL uses execution signals; and Re:Form applies verifier-guided RL to individual Dafny proofs (Wei et al., 2025; Pinto et al., 2026; Le et al., 2022; Yan et al., 2025). Unlike whole-output verifier rewards, CRAFT trains toward a filtered clause pool by salvaging the inductive part of an imperfect response and rewarding its strength and complementary group coverage. This distinction matters because verifiable-reward RL can improve small-budget accuracy without expanding broad-sampling coverage (Yue et al., 2025).

4 METHOD

CRAFT couples an inference protocol that proves targets by composing clauses across responses with a training pipeline that optimizes that protocol without ever seeing the target. Three design decisions organize this section: (i) *target hiding*—generation, distillation, and reward computation all operate on the masked loop, and the target is restored only for the final proof (Section 4.2); (ii) *the clause, not the response, is the credit unit* that survives filtering and earns reward (Section 4.1); (iii) *training mirrors inference*—credit rewards complementary coverage across a response group, because inference unions that group (Section 4.5). The guiding thesis is that because inference composes, training should optimize composability: raising the compose@k curve and reaching its large-pool performance with fewer rollouts.

4.1 ROLLOUT–DECOMPOSE–FILTER–COMPOSE

For a target-masked loop $\mathcal{L}$, the model emits ACSL loop-invariant annotations (Baudin et al., 2021). Parsing extracts each `loop invariant ...;` annotation as one clause; normalization turns response i into the clause sequence $A_i = \langle a_{i1}, \ldots, a_{im_i} \rangle$. We cap it at K clauses (K=20; Appendix C), $\bar{A}_i = \langle a_{i1}, \ldots, a_{i,\min(m_i,K)} \rangle$, with overflow $o(A_i) = \max(0, m_i - K)$. Extraction makes the annotation, rather than the complete response, the unit that can survive filtering and earn credit. We write $\mathrm{set}(\bar{A}_i)$ for the underlying clause set when applying set operations.

Algorithm 1 Rollout–decompose–filter–compose

Input: masked loop $\mathcal{L}$, parsed clause sequences $A_{1:G}$, cap K
Output: filtered clause set C

$C \leftarrow \bigcup_i \mathrm{set}(\mathrm{Cap}_K(\mathrm{Normalize}(A_i)))$
$C \leftarrow \mathrm{ConservativeDedup}(C)$
repeat
  $C_{\mathrm{parse}} \leftarrow \mathrm{ParserErrors}(\mathcal{L}, C)$; $C \leftarrow C \setminus C_{\mathrm{parse}}$
until $C_{\mathrm{parse}} = \varnothing$ or $C = \varnothing$
repeat
  $C_{\mathrm{drop}} \leftarrow \{c \in C : \neg\, \mathrm{WP}_{\mathrm{est,pres}}(\mathcal{L}, C, c)\}$; $C \leftarrow C \setminus C_{\mathrm{drop}}$
until $C_{\mathrm{drop}} = \varnothing$ or $C = \varnothing$
return C

Given G responses, the pipeline unions their clauses,

$$C_G^{\mathrm{pool}} = \bigcup_{i=1}^{G} \mathrm{set}(\bar{A}_i),$$

then iteratively removes syntax failures and applies Houdini to delete clauses whose establishment or preservation obligations fail (Flanagan & Leino, 2001). We write the resulting inductive subset as $\mathsf{Filter}_{\mathcal{L}}(C_G^{\mathrm{pool}})$. Conjoining valid clauses preserves validity and can compose a bound from one response with a relation from another; filtering keeps such clauses from otherwise imperfect responses but cannot invent a missing relation. Algorithm 1 computes $\mathsf{Filter}_{\mathcal{L}}$; we write its fixed point as $\mathsf{H}_{\mathcal{L}}(C)$ and the conjunction of a clause set C as $I_C = \bigwedge_{c \in C} c$. The algorithm is shared by SFT synthesis and inference.

The following statements formalize the composition used by the pipeline; all proofs are deferred to Appendix B.

Proposition 1 (Closure and strengthening under conjunction)**.** *Let $I_1, \ldots, I_m$ be valid invariants of the same loop, for finite $m \geq 1$. Then $I = \bigwedge_{j=1}^{m} I_j$ is valid and implies every I_j, i.e., $I \preceq I_j$. Moreover, $I \prec I_j$ exactly when $I_j \not\Rightarrow \bigwedge_{\ell \neq j} I_\ell$.*

Proposition 2 (Mutually supported composition)**.** *Let $C = \{c_1, \ldots, c_m\}$ be any finite set of candidate clauses. The conjunction I_C is a valid invariant whenever*

$$P \Rightarrow I_C, \qquad \forall j,\ \{I_C \wedge B\}\, S\, \{c_j\}.$$

Individual obligations $\{c_j \wedge B\}\, S\, \{c_j\}$ need not hold: preservation may be supplied by any number of companion clauses in C. Mutual support cannot, however, repair a clause that fails establishment from P. The resulting valid invariant satisfies $I_C \preceq c_j$, with $I_C \prec c_j$ whenever $c_j \not\Rightarrow I_{C \setminus \{c_j\}}$.

Theorem 3 (Candidate-relative optimality of ideal Houdini)**.** *For a finite parser-admitted pool C, assume exact decisions for all establishment and preservation obligations in Algorithm 1. Its fixed point $\mathsf{H}_{\mathcal{L}}(C)$ is jointly inductive, and every jointly inductive subset $C' \subseteq C$ satisfies $C' \subseteq \mathsf{H}_{\mathcal{L}}(C)$. Consequently, $I_{\mathsf{H}_{\mathcal{L}}(C)} \preceq I_{C'}$: the result is the strongest conjunction expressible as a subset of C. This maximality is relative to the supplied pool and the exact-oracle assumption.*

This is the classic Houdini greatest-fixpoint property (Flanagan & Leino, 2001), restated in our clause notation. In practice the per-obligation prover budget (Section 5.1) makes this oracle approximate.

RL scoring additionally removes clauses contradicted by observed reachable states $\mathcal{P}$ before Houdini:

$$\mathsf{PF}_{\mathcal{P}}(C) = \{c \in C : \forall p \in \mathcal{P},\ p \models c\}.$$

This positive filter is a cheap refutation step used only for RL scoring, not a validity proof; Frama-C/WP remains the authority. Appendix C gives normalization and filtering details.

4.2 Inference Objective

At inference, the pipeline samples k masked responses and produces

$$C_k^{\text{pool}} = \bigcup_{i=1}^{k} \text{set}(\bar{A}_i), \qquad C_k = \mathsf{Filter}_{\mathcal{L}}(C_k^{\text{pool}}), \qquad I_k = I_{C_k}.$$

Only then is Q restored. The target utility and inference objective are

$$U_Q(A_{1:k}) = \mathbf{1}[\text{Verify}(\mathcal{L}, I_k, Q)], \qquad \max_{\pi} \mathbb{E}_{A_{1:k}\sim\pi(\cdot|\mathcal{L})}[U_Q(A_{1:k})].$$

Here Verify requires establishment, preservation, and every restored target at its original location; for one post-loop target, its last obligation is $I_k \wedge \neg B \Rightarrow q$. Its empirical counterpart is compose@k, which can succeed even when no standalone response is sufficient. The training sections below therefore optimize a target-hidden surrogate for this curve.

4.3 Target-Independent Strength Examples

To grade inductive sets without Q, we execute a deterministic, precondition-checked input sweep and record reachable loop-head states $\mathcal{P}$. From these executions we construct two negative-example families: local one- or two-variable perturbations that break relations, and continuations of the real body after a witnessed loop exit. Cleaning is deliberately conservative—any candidate that executions cannot rule out is dropped: it removes observed reachable states, possible fresh entries, duplicates, and candidates involving unsupported state. These examples are training signals, not proofs of global unreachability; unsupported or incomplete executions disable the affected family. Figure 5, Algorithm 2, and exact budgets appear in Appendix C.1.

Let $\mathcal{N}$ be the retained negative traces. For a retained clause set C, $\text{rej}(C) \subseteq \mathcal{N}$ contains each trace on which at least one clause in C is false at some recorded state; singleton traces recover ordinary state rejection.

4.4 Distilling Multi-Response Search

For each masked loop $\mathcal{L}$ in the training corpus $\mathcal{D}_{\text{train}}$, let $C_{\mathcal{L}}^{\text{pool}}$ be a sampled response-group union and $C_{\mathcal{L}}^{\text{sft}} = \mathsf{Filter}_{\mathcal{L}}(C_{\mathcal{L}}^{\text{pool}})$. We serialize every nonempty result as one SFT target:

$$\mathcal{D}_{\text{SFT}} = \{(\mathcal{L}, \text{serialize}(C_{\mathcal{L}}^{\text{sft}})) : \mathcal{L} \in \mathcal{D}_{\text{train}},\ C_{\mathcal{L}}^{\text{sft}} \neq \varnothing\}.$$

Neither input nor target contains Q: SFT serializes a Houdini-retained multi-response search result as one response.

4.5 Clause-Decomposed, Group-Compositional Reward

The reward design responds directly to the sharpening tendency described in Section 1. Group-relative optimization with a whole-response reward concentrates probability mass on a few high-reward responses, eroding the clause diversity that the pipeline composes at inference. We therefore decompose credit *within* each rollout—surviving clauses earn reward even when siblings fail—and re-allocate credit *across* the group by each rollout's marginal contribution to pooled coverage, paying the policy for expanding the joint clause pool rather than for reproducing its most common clauses.

For rollout i, define its verifier-retained subset and rejected negatives as

$$C_i^{\text{pf}} = \mathsf{PF}_{\mathcal{P}}(\text{set}(\bar{A}_i)), \qquad C_i^{\text{train}} = \mathsf{Filter}_{\mathcal{L}}(C_i^{\text{pf}}), \qquad R_i = \text{rej}(C_i^{\text{train}}).$$

This projection implements *intra-rollout decomposition*: malformed or non-inductive clauses are removed without erasing useful clauses in the same response. When retained negatives exist, the reward adds no bonus merely for valid syntax or whole-response validity. The subset receives the dense, target-independent strength score

$$\text{cov}(A_i) = \frac{|R_i|}{|\mathcal{N}|}.$$

This and the following allocation apply when $\mathcal{N} \neq \varnothing$; when the sampler retains none, we use the binary fallback in Appendix C.2.

Independent coverage, however, rewards a complementary discovery exactly like one repeated by every peer. We therefore allocate credit for the union $\bigcup_i R_i$ across the rollout group $\mathcal{G} = \{1, \ldots, G\}$ as a coverage game. Let $f(n) = \sum_{j=1}^{G} \mathbf{1}[n \in R_j]$ count how many peers reject trace n. The Shapley value of rollout i has the closed form (Shapley, 1953) (proof in Appendix B)

$$\phi_i = \frac{1}{|\mathcal{N}|} \sum_{n \in R_i} \frac{1}{f(n)}, \qquad \sum_{i=1}^{G} \phi_i = \frac{|\bigcup_i R_i|}{|\mathcal{N}|}. \tag{1}$$

Thus a unique rejection receives full credit, whereas $f(n)$ redundant rejections split it. The Shapley allocation is the unique credit rule satisfying efficiency, symmetry, linearity, and the null-player axiom (Shapley, 1953); unlike a leave-one-out marginal, which credits only rejections supplied by no peer and does not account for the full union, it splits credit for shared rejections instead of discarding it. This closed form uses already-computed rejection sets and adds no verifier calls. It implements *inter-rollout composition*: the same response earns less group credit when its verified constraints are already supplied by peers.

With the within-response duplicate count $d_{\text{dup}}(A_i) = |\bar{A}_i| - |\operatorname{dedup}(\bar{A}_i)|$ (clauses identical after normalization), the reward is

$$r_i = w_c \operatorname{cov}(A_i) + w_g \phi_i - w_d d_{\text{dup}}(A_i) - w_o o(A_i), \tag{2}$$

where $(w_c, w_g, w_d, w_o) = (1.0, 0.3, 0.02, 0.05)$; Section 5.6 ablates each term. GRPO normalizes these group-conditioned rewards within the sampled response group. Shapley credit distinguishes equal-coverage rollouts when their rejection sets overlap differently, favoring less redundant coverage. It remains a target-independent behavioral surrogate rather than credit for final Q-sufficiency. Each training step samples a group of G responses per masked loop, scores them by Equation (2), and applies one GRPO update; this training-time group is distinct from the inference sweep over k (Section 4.2). Appendix C.2 gives the standard policy objective, duplicate normalization, and fallback behavior.

5 Experiments

We ask how composition scales with sampling (RQ1), how much each pipeline component contributes (RQ2), how CRAFT compares with specialized tools (RQ3), how much RL adds (RQ4), and what drives the training gain (RQ5).

5.1 Experimental Setup

Data and judge. Training data comprise 5,000 RL records and 14,858 SFT records of target-masked scalar-integer single-loop programs; SFT targets are synthesized with `gpt-5.6-luna` and verified by Frama-C/WP (Appendix G). Evaluation uses the standard benchmark suites of prior learning-based invariant-generation systems: 832 C programs, comprising 316 linear programs (133 Code2Inv (Si et al., 2020), 84 SyGuS 2019 (Alur et al., 2013), 99 SV-COMP 2024 (Beyer, 2023)) and 50 nonlinear-arithmetic (NLA) programs (30 LIPuS (Yu et al., 2023), 20 SV-COMP 2024) as released with Clause2Inv (Cao et al., 2025), plus 466 integer programs from Loopy (Kamath et al., 2023). The training–evaluation overlap audit is in Appendix F. We remove non-contract comments and mask all target predicates before generation, then restore the untouched source only after invariants are fixed. Success requires Frama-C/WP (Kirchner et al., 2015), under a 5-second per-obligation prover budget, to prove establishment, preservation, and every restored target; all failures count as zero.

Inference protocol. All CRAFT evaluation runs share target masking, the canonical prompt, clause processing, and Houdini; external tools retain their own target-hidden orchestration. Within CRAFT, API models use `reasoning_effort=none`; all locally served checkpoints use non-thinking decoding to match data synthesis and training. Decoding uses temperature 1.0 and top-p 1.0. API calls cap completions at 8,192 tokens, including provider-internal reasoning tokens; local vLLM calls cap each response at 2,048 emitted tokens. We use no re-roll or target feedback. RQ1 reuses one saved batch of 100 stochastic responses per task. Appendix H gives benchmark provenance, serving, and training details.

Metrics. Throughout evaluation, success means Q-sufficiency under the final verifier. pass@k is the probability that at least one of k responses proves Q. Whenever a saved pool contains c successful responses among $n = 100$, we use the unbiased estimator $\widehat{\text{pass@}}\,k = 1 - \binom{n-c}{k} / \binom{n}{k}$ for that program and average it over programs. Only responses that prove Q count as successes. compose@k is the fraction of programs proved by unioning the first k responses and applying Houdini. pass@k requires one response to contain the whole proof, whereas compose@k can compose clauses from different responses. We do not compute the exact all-subset analogue for compose@k, because it would require filtering $\binom{100}{k}$ unions per program; we evaluate the fixed saved prefix instead. For the measured budget grid $\mathcal{K} = \{1, 4, 8, 16, 32\}$, we report $k_{95} = \min\{k \in \mathcal{K} : \text{Comp}_\pi(k) \geq 0.95\ \text{Comp}_\pi(32)\}$, where $\text{Comp}_\pi(k)$ is the measured compose@k curve. A smaller k_{95} means the retained pool reaches its large-budget performance sooner.

5.2 RQ1: How Do pass@k and compose@k Scale?

We sample 100 target-hidden responses per task from six base checkpoints: Qwen3-0.6B, Qwen3-4B, Qwen3-8B, Qwen3-14B, Qwen3-30B-A3B (Yang et al., 2025), and Llama 3.1 8B. Complete values and k_{95} appear in Appendix 8, per-stratum breakdowns in Table 9.

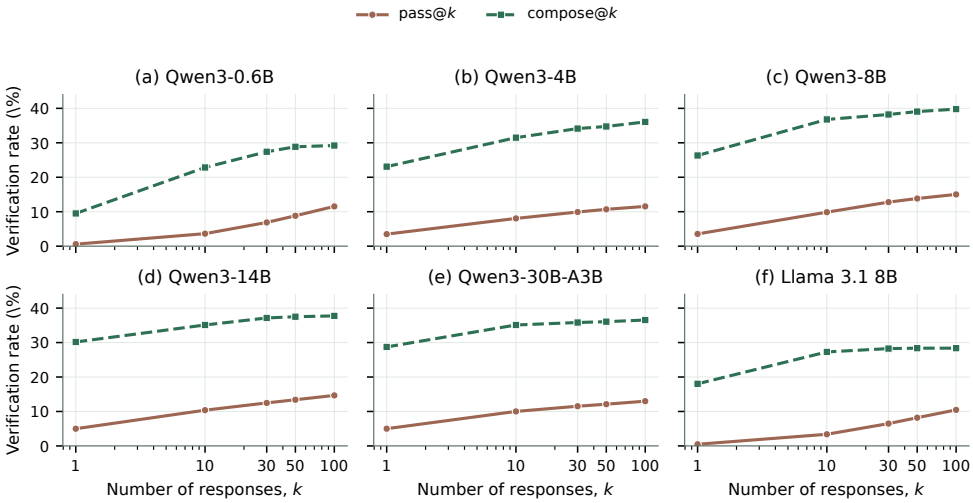

Figure 2: pass@k (unbiased estimate) and prefix compose@k. Composition saturates by k =TBD, while pass@k keeps growing.

At $k = 1$, prefix compose@k is 8.92–25.18 points above the unbiased pass estimate across checkpoints. Because these are different estimands rather than paired first-response outcomes, we treat this gap as descriptive; RQ2 reports the paired effect of processing the same response. In the retained response pools, every model reaches 95% of compose@32 by k =TBD, with at most TBD additional points through $k = 32$. In contrast, pass@8 reaches only TBD of pass@32 on the five Qwen checkpoints and gains another TBD points by $k = 32$. Llama 3.1 8B is harder in standalone proof generation—pass@32 is TBD—but shows the same compositional pattern: compose rises from 18.03% at $k = 1$ to TBD at $k = 16$, then changes by only TBD points. Composition thus finds most of its useful clauses within the first few dozen samples, whereas standalone proofs require wider sampling. The compose@32 values (TBD) further suggest that these pools remain limited by clause quality or diversity: extending the budget to 32 responses does not remove the gap.

Insight. pass@1 is not an appropriate primary metric for compositional invariant generation. By treating a response as indivisible, it assigns zero both to useless outputs and to incomplete outputs containing valid, reusable clauses. It remains a standalone diagnostic, whereas compose@k directly measures whether clauses found within the inference budget form a sufficient invariant after filtering.

5.3 RQ2: What Does Each Component Contribute?

The main neural experiment evaluates saved responses per program at budgets k=1 and k=8 and decomposes the pipeline into cumulative stages. Framework-free rows judge each response as one indivisible conjunction (pass@k); pipeline rows decompose the same responses into clauses, filter them, and compose the survivors (compose@k). The three Qwen3 models add the training stages (+SFT, +SFT+RL) and two ablation rows that repeat the trained checkpoints without the pipeline; Llama 3.1 8B reports the SFT rows without RL.

	Linear / 316		NLA / 50		Loopy / 466		All / 832	
Stage	k=1	k=8	k=1	k=8	k=1	k=8	k=1	k=8
Qwen3-4B								
bare (pass)	2.80	TBD	0.00	TBD	4.34	TBD	3.49	TBD
+pipeline (compose)	23.42	TBD	2.00	TBD	25.11	TBD	23.08	TBD
+SFT	26.27	TBD	2.00	TBD	28.54	TBD	26.08	TBD
+SFT+RL	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
SFT, no pipeline (pass)	6.89	TBD	0.10	TBD	8.53	TBD	7.40	TBD
SFT+RL, no pipeline (pass)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-8B								
bare (pass)	2.79	TBD	0.00	TBD	4.44	TBD	3.55	TBD
+pipeline (compose)	25.63	TBD	2.00	TBD	29.40	TBD	26.32	TBD
+SFT	29.11	TBD	2.00	TBD	25.75	TBD	25.60	TBD
+SFT+RL	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
SFT, no pipeline (pass)	6.53	TBD	0.02	TBD	8.25	TBD	7.10	TBD
SFT+RL, no pipeline (pass)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-14B								
bare (pass)	4.36	TBD	0.12	TBD	5.94	TBD	4.99	TBD
+pipeline (compose)	31.01	TBD	0.00	TBD	32.83	TBD	30.17	TBD
+SFT	31.01	TBD	2.00	TBD	31.12	TBD	29.33	TBD
+SFT+RL	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
SFT, no pipeline (pass)	7.70	TBD	0.22	TBD	9.51	TBD	8.26	TBD
SFT+RL, no pipeline (pass)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Llama 3.1 8B								
+pipeline (compose)	16.77	TBD	4.00	TBD	20.39	TBD	18.03	TBD
+SFT	26.27	TBD	0.00	TBD	25.11	TBD	24.04	TBD
SFT, no pipeline (pass)	5.41	TBD	0.04	TBD	5.48	TBD	5.12	TBD

Table 1: Cumulative contribution of each pipeline component, by benchmark stratum (% verified, `reasoning_effort=none`/non-thinking). Framework-free rows report pass@k; pipeline rows report compose@k on the same saved responses, so adjacent rows isolate one component. RL is trained on the three Qwen3 models; Llama 3.1 8B reports SFT rows only. Complete probe curves for all six models appear in Appendix 8 and 9. API models under the same protocol appear in Table 16 (Appendix I.5).

Because framework-free and pipeline rows reuse the same saved responses, their paired difference isolates the net effect of clause extraction, normalization, and Houdini without additional model calls: composing lifts every model by 19.6–25.2 points at k=1 and by TBD points at k=8 in aggregate. Per stratum the lift is positive everywhere except NLA under Qwen3-14B, where composing trails pass@k (0.12→0.00 at k=1, TBD at k=8)—the large-model NLA anomaly of Table 9. All rows report full-workload means over the same 832 programs. Per-model probe curves appear in Appendix 8.

5.4 RQ3: How Does CRAFT Compare with Specialized Tools?

We compare against the target-hidden pipelines of AutoSpec, SESpec, and Clause2Inv, and against Daikon 5.8.24 (Wen et al., 2024; Yang et al., 2026b; Cao et al., 2025; Ernst et al., 2007). All tool rows use `gpt-5-nano` where they call an LLM. The Naive row measures the model's unaided behavior:

one call with no invariant-specific instructions, under which the model emits only the few clauses it considers safe. CRAFT's prompt instead adapts the model to the pipeline, asking for up to twenty clauses that cover conservation laws, progress bounds, and exit facts. This trades single-response reliability for clause coverage: pass@1 judges all clauses in a response as an indivisible conjunction, so one invalid or non-inductive clause fails the whole response, and CRAFT's richer raw outputs score below Naive before filtering (3.61% vs. 16.47%). The gap is by design—the discarded clauses carry no cost, because the pipeline keeps only what survives filtering, which is what compose@1 measures. Clause2Inv's unsupported inputs remain failures in the common 832-program denominator. Budgets are fixed before comparison but not matched: each system retains its predeclared native orchestration.

System	Linear / 316	NLA / 50	Loopy / 466	All / 832	Tokens / task	Time / task
AutoSpec	47.78%	6.00%	42.27%	42.19%	2,181.72	27.34 s
SESpec	28.80%	4.00%	33.05%	29.69%	22,081.61	68.50 s
Clause2Inv	20.89%	0.00%	0.00%	7.93%	1,056.25	8.41 s
Daikon	23.10%	0.00%	21.67%	20.91%	0	4.89 s
Loopy	20.25%	0.00%	19.74%	18.75%	14,513.37	147.16 s
Naive	15.19%	2.00%	18.88%	16.47%	225.95	2.98 s
CRAFT (pass@1)	2.53%	0.00%	4.72%	3.61%	1,135.83	7.86 s
CRAFT (compose@1)	34.81%	22.00%	34.55%	33.89%	TBD	TBD
CRAFT (compose@4)	48.10%	30.00%	51.72%	49.04%	TBD	TBD
CRAFT (compose@8)	55.38%	44.00%	56.87%	55.53%	TBD	TBD

Table 2: Common target-hidden comparison with `reasoning_effort=none`, using the same untouched inputs and Frama-C/WP final judge. All LLM-driven rows share the same fixed backbone model; tokens and time are mean totals per task. The three compose rows are recomputed from one archived ten-rollout pool by prefix-subset composition, unioning the first k responses through the same filter chain.

The archived reasoning-enabled comparison appears in Appendix 18. On four further API models the same pairing lifts compose@1 over pass@1 by 3.8–23.4 points (mean 16.6; Appendix 16), on top of the backbone's 30.3: the gain is a property of the pipeline, and it compresses as single-shot accuracy rises (GPT-5.6-luna already passes 81.3% of tasks in one shot). Under this setting, increasing CRAFT from four to eight candidates raises verification from 49.0% to 55.5%.

Failure modes under target hiding. The specialized systems degrade for different structural reasons. In the evaluated one-round configuration, AutoSpec requests eight completions in parallel, pools their clauses, and verifier-filters the union; it is therefore already an eight-proposal composition method rather than a single-proposal baseline. Nevertheless, it verifies 351 programs, 23 fewer than CRAFT compose@1. Its extra samples mainly add redundant or weak clauses rather than the complementary facts recovered by CRAFT's clause processing. SESpec instead makes 5.09 sequential calls per completed task on average. This refinement-heavy schedule does not provide the independent parallel exploration needed when the target is absent; moreover, errors from the target-hidden verifier diagnose syntax and inductiveness, not which missing relation would establish the hidden goal. Clause2Inv's ICE loop normally uses false-state (`F:`) counterexamples induced by the postcondition. Target-hidden execution suppresses the postcondition check and retains only pre-state and transition counterexamples, removing its principal goal-directed feedback signal; its required transition VCs are also unavailable for all 466 Loopy programs.

Loopy exhibits a separate interface failure. Its extractor accepts only the first valid Markdown-fenced code block. Although 98.8% of the non-reasoning initial responses contain textual `loop invariant` clauses, only 1.75% use the required fence and reach Houdini; for repair responses the corresponding rates are 94.5% and 1.13%. Houdini can prune bad clauses but cannot recover discarded ones. Target hiding further removes the assertion-specific checker diagnostic expected by Loopy's repair prompt, leaving only generic inductiveness feedback: repair adds just 15 successes to the 141 initially solved tasks, for 156/832 (18.75%) overall. Thus this row primarily measures a brittle parser and a repair protocol mismatched to the hidden-target setting, rather than the quality of

the invariants present in the raw responses. The archived medium-reasoning run is reported separately in the appendix.

Finding. Target visibility is a problem boundary, not an implementation detail. Goal-directed feedback—Clause2Inv's postcondition-induced counterexamples, Loopy's assertion-specific diagnostics—vanishes once the target is hidden, and the two systems fall to 7.93% and 18.75%; four target-hidden responses composed by CRAFT verify TBD. Under target hiding, complementary composition survives where iterative goal-directed repair does not.

5.5 RQ4: How Much Does Reinforcement Learning Improve Inference?

We evaluate matched Qwen3-8B before–after RL paths from base and SFT initializations. Inference is fixed within each pair; RQ1 is not used to estimate the RL effect.

What to expect. If RL with whole-response credit merely sharpens the distribution (Yue et al., 2025), it should raise pass@1 while leaving or degrading large-k coverage. Clause-decomposed, group-compositional credit predicts the opposite pattern: pass@k stays flat, while compose@k shifts up at small budgets without degrading at large ones.

The SFT stage distills the filtered multi-response union into single responses (Section 4.4). On Qwen3-8B it doubles pass@1 (3.55%→7.10%) and lifts compose@8 from TBD to TBD, while compose@1 stays flat (26.32%→25.60%): the distilled clauses help once several responses pool, not in isolation (Table 3); full SFT curves across four backbones appear in Appendix I.2.

Initialization	Training stage	pass@1	compose@1	compose@4	compose@32	k_{95}
Base 8B	before RL (Zero)	3.55%	26.32%	TBD	TBD	TBD
Base 8B	after RL (RL-Zero)	8.77%	53.85%	63.82%	71.51%	16
SFT 8B	before RL (SFT)	7.10%	25.60%	TBD	TBD	TBD
SFT 8B	after RL (SFT+RL)	TBD	TBD	TBD	TBD	TBD

Table 3: Matched Qwen3-8B evaluation before and after RL.

Results. Consistent with the redistribution view, response-level accuracy does not improve after RL (Figure 3, left; pass@1 TBD vs. TBD). The redistribution is instead steered toward the complementary tail of the clause pool: compose@k^* with k^*=TBD matches its initialization's compose@32 (TBD), an effective TBD-fold budget reduction, and compose@32 itself does not degrade (TBD vs. TBD). RL thus packs into k^* samples the useful clauses its initialization produces only across thirty-two. A clause-frequency analysis against TBD initialization samples attributes part of the gain to clauses the initialization produces with probability below TBD per response.

5.6 RQ5: What Drives the Training Gain?

Both ablations train Qwen3-8B from the Zero initialization on the curated pool with all other training and inference settings fixed; runs within an ablation differ only in the reward or the negative sampler. Appendix I.4 gives the exact reward definitions and complete numbers.

Reward-function ablation. Variants progress from binary inductiveness to whole-response coverage, then credit surviving clauses individually; the full reward additionally adds Shapley credit and penalizes duplicate clauses. All use a 20-clause cap; coverage variants share the overflow cost, while the binary control remains $\{0, 1\}$.

Figure 4 reveals two effects. *First, response-level credit collapses composition.* Binary inductiveness degenerates into repeating a few safe clauses: compose@1 falls 17.6 points below the untrained model. Whole-rollout coverage sharpens harder still—the best pass@k at every budget—yet its compose curve gains only 5.4 points from one union to thirty-two. This is the sampling-limit failure mode anticipated in Section 5.5. *Second, credit must be decomposed to clauses.* Clause-decomposed

`rl_panels.pdf` placeholder — generated by `figures/plot_rl_panels.py` from the canonical RL pool artifact (pass@k left, compose@k right with the SFT compose@32 reference line and the crossing budget k^* annotated).

Figure 3: Response-level vs. compositional effects of RL (Qwen3-8B, SFT initialization). (a) pass@k is expected to remain flat, consistent with redistribution rather than support expansion (Yue et al., 2025). (b) compose@k shifts upward at small budgets: the RL policy reaches its initialization's compose@32 (dotted line) at k^* samples, and compose@32 itself does not degrade.

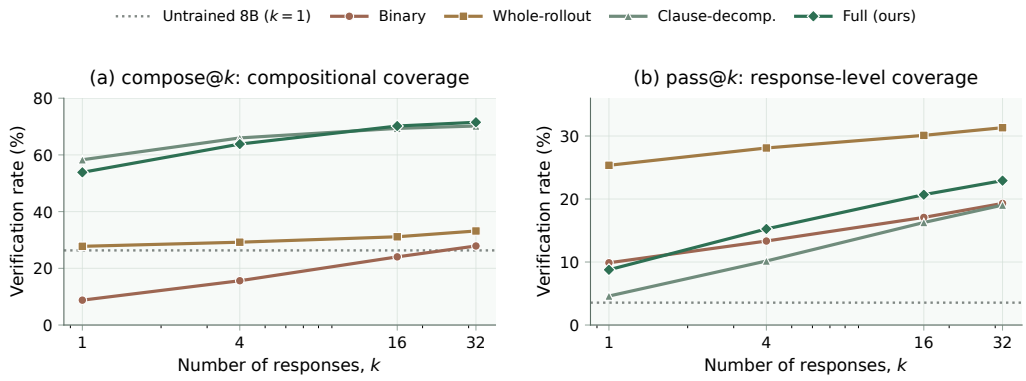

Figure 4: Reward-function ablation on Qwen3-8B (Zero initialization). Response-level credit (Binary, Whole-rollout) raises pass@k (b) but flattens compose@k (a); clause-decomposed credit reverses the trade-off, and the full compositional credit is best at large budgets. Dotted lines: the untrained model at k=1. Exact numbers in Table 14 (Appendix I.4).

coverage more than doubles compose@1 (27.76%→58.29%) and lifts compose@32 from 33.17% to 70.19%; the full compositional credit is best at large budgets (71.51% at k=32) while keeping a higher pass@1.

Breaking the response-level sampling limit. Why does credit structure decide the outcome? Whole-response rewards grade each rollout by its own clauses alone, so they can only push probability mass toward responses the policy already favors (Yue et al., 2025). Group-compositional credit instead grades each rollout against its peers, making the objective the group's union rather than any single response: the compose curve more than doubles at every budget while pass@1 still improves. What RL does to the sampling distribution is decided by what the reward values, not by RL itself.

Finding (RLVR limits). Response-level RLVR sharpens rather than expands the sampled support: it lifts pass@1 but leaves composition nearly flat. Clause decomposition supplies denser credit and reverses this: compose@32 rises from 33.17% to 70.19% (71.51% with the full compositional credit).

Negative-sampling ablation. Structured sampling matches or exceeds random perturbations at every budget, improving compose@1 and compose@32 by 1.8 and 2.8 points (Appendix I.4).

6 Conclusion and Limitations

CRAFT makes filtered clause composition the inference target: its pipeline composes rollouts, SFT distills their union, and target-independent RL rewards useful clauses and complementary coverage.

The recipe transfers beyond loop invariants. It applies to problems whose answers (i) decompose into independently checkable units composed by conjunction, (ii) admit a cheap mechanical filter for unit validity, (iii) include valid-but-weak degenerate solutions that pass/fail cannot grade, and (iv) allow behavioral negatives, on which weak and strong solutions disagree, to be derived from executions or simulations. Test generation judged by mutant killing, auxiliary-lemma synthesis for proof assistants, and data-constraint discovery all satisfy these conditions.

Evidence is limited to scalar-integer, single-loop C programs, Frama-C/WP and ACSL, and vLLM-served local checkpoints. Credit also filters rollouts separately whereas inference filters their union. Appendices J and L detail these scope and reward approximations.

AI Use Statement

Generative AI tools are integral to the evaluated system: they produce the invariant candidates and synthetic SFT data described in the paper. They were also used during research to refine hypotheses, methodology, and experiments; formulate and check mathematical claims; implement and test software; analyze and interpret experimental artifacts; prepare figures and tables; search and summarize literature; and draft, edit, and translate manuscript text. The authors reviewed all AI-assisted work and tested generated code. They take responsibility for independently validating the final content, empirical claims, and released artifacts.

Reproducibility Statement

The method and objective are specified in Section 4; Appendices C–H give the implementation defaults, prompt, training interface, data-overlap audit, and evaluation protocol. Appendix I organizes the available measurements, including per-seed results.

References

Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, pp. 1–8, 2013.

Patrick Baudin, Pascal Cuoq, Jean-Christophe Filliâtre, Claude Marché, Benjamin Monate, Yannick Moy, and Virgile Prevosto. ACSL: ANSI/ISO C specification language. `https://frama-c.com/html/acsl.html`, 2021.

Dirk Beyer. Competition on software verification (SV-COMP). `https://sv-comp.sosy-lab.org/`, 2023.

Weining Cao, Guangyuan Wu, Tangzhi Xu, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. Clause2Inv: A generate-combine-check framework for loop invariant inference. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1009–1030, 2025. doi: 10.1145/3728920.

Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP*, pp. 9164–9175, 2023. doi: 10.18653/v1/2023.findings-emnlp.614.

Michael A. Colón, Sriram Sankaranarayanan, and Henny B. Sipma. Linear invariant generation using non-linear constraint solving. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 420–432. Springer, 2003.

Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.

Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69:35–45, 2007.

Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *Proc. 31st ACM Joint European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE)*, pp. 1229–1241, 2023.

Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc. Int. Symp. of Formal Methods Europe (FME)*, pp. 500–517. Springer, 2001.

Robert W. Floyd. Assigning meanings to programs. In *Proc. Symp. in Applied Mathematics: Mathematical Aspects of Computer Science*, volume 19, pp. 19–32, 1967.

Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A robust framework for learning invariants. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 69–87. Springer, 2014.

C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12 (10):576–580, 1969.

Zhechong Huang, Zhao Zhang, Zeyu Sun, Huifeng Sun, and Yingfei Xiong. Reinforcement learning with negative tests as completeness signal for formal specification synthesis. *arXiv preprint arXiv:2604.05820*, 2026.

Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.

Michael Karr. Affine relationships among variables of a program. *Acta Informatica*, 6(2):133–151, 1976.

Zachary Kincaid, Jason Breck, Ashkan Forouhi Boroujeni, and Thomas Reps. Compositional recurrence analysis revisited. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 248–262, 2017.

Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-c: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Chang Liu, Xiwei Wu, Yuan Feng, Qinxiang Cao, and Junchi Yan. Towards general loop invariant generation: A benchmark of programs with memory manipulation. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2024a.

Ruibang Liu, Minyu Chen, Ling-I Wu, Jingyu Ke, and Guoqiang Li. Enhancing automated loop invariant generation for complex programs with large language models. *arXiv preprint arXiv:2412.10483*, 2024b.

Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proc. Int. Conf. on Software Engineering (ICSE)*, 2025.

Kenneth L. McMillan. Interpolation and SAT-based model checking. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 1–13. Springer, 2003.

Ido Pinto, Yizhak Yisrael Elboher, Haoze Wu, Nina Narodytska, and Guy Katz. Not all invariants are equal: Curating training data to accelerate program verification with SLMs. *arXiv preprint arXiv:2603.15510*, 2026.

Gabriel Ryan, Justin Wong, Jianan Yao, Ronghui Gu, and Suman Jana. CLN2INV: Learning loop invariants with continuous logic networks. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2020.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Lloyd S. Shapley. A value for n-person games. In Harold W. Kuhn and Albert W. Tucker (eds.), *Contributions to the Theory of Games II*, volume 28 of *Annals of Mathematics Studies*, pp. 307–317. Princeton University Press, 1953.

Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants for program verification. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, pp. 7751–7762, 2018.

Xujie Si, Aaditya Naik, Hanjun Dai, Mayur Naik, and Le Song. Code2Inv: A deep learning framework for program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 151–164. Springer, 2020.

Anjiang Wei, Tarun Suresh, Tianran Sun, Haoze Wu, Ke Wang, and Alex Aiken. InvBench: Can LLMs accelerate program verification with invariant synthesis? *arXiv preprint arXiv:2509.21629*, 2025.

Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. Enchanting program specification synthesis by large language models using static analysis and program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 302–328. Springer, 2024.

Guangyuan Wu, Weining Cao, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. LLM meets bounded model checking: Neuro-symbolic loop invariant inference. In *Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2024a.

Haoze Wu, Clark Barrett, and Nina Narodytska. Lemur: Integrating large language models in automated program verification. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024b.

Chuanhao Yan, Fengdi Che, Xuhan Huang, Xu Xu, Xin Li, Yizhi Li, Xingwei Qu, Jingzhe Shi, Chenghua Lin, Yaodong Yang, Binhang Yuan, Hang Zhao, Yu Qiao, Bowen Zhou, and Jie Fu. Re:Form—reducing human priors in scalable formal software verification with RL in LLMs: A preliminary study on Dafny. *arXiv preprint arXiv:2507.16331*, 2025.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Fanpeng Yang, Xing Li, Shuling Wang, Jie An, Zeyu Sun, Shenghua Feng, Wenhan Wang, Weiyi Wang, Naijun Zhan, and Fanjiang Xu. How powerful are LLMs in generating formal program specifications? In *Proc. Int. Conf. on Machine Learning (ICML)*, 2026a.

Fanpeng Yang, Xu Ma, Shuling Wang, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu. Integrating symbolic execution with LLMs for automated generation of program specifications. *arXiv preprint arXiv:2506.09550*, 2026b.

Jianan Yao, Gabriel Ryan, Justin Wong, Suman Jana, and Ronghui Gu. Learning nonlinear loop invariants with gated continuous logic networks. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 106–120, 2020.

Shiwen Yu, Ting Wang, and Ji Wang. Loop invariant inference through SMT solving enhanced reinforcement learning. In *Proc. ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, pp. 175–187, 2023. doi: 10.1145/3597926.3598047.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*, volume 38, 2025.

A Scope and Motivating Example

We focus on numerical loops because their arithmetic equalities and inequalities, including polynomial relations, admit scalable automatic checking. This controlled setting factors out arrays, heaps, recursive data structures, auxiliary functions, and inductive predicates, allowing the study to isolate algebraic discovery and loop dynamics. Concretely, we consider one braced `while` loop over scalar C integers, including nonlinear updates. Even this restricted setting can hide nonlinear relations across coupled recurrences. For example:

```c
void f(int k, int z) {
  int x = 1, y = 1, c = 1;
  while (c < k) { c++; x = x*z + 1; y = y*z; }
  /*@ assert 1+x*z-x-z*y == 0; */
}
```

When visible, the assertion can be copied verbatim as an invariant; when hidden, its nonlinear relation must be recovered from the coupled updates.

B Proofs of Compositionality

This section contains all proofs for the formal composition claims in Section 4.1. Write $\mathcal{L} = (P, B, S)$, where P denotes loop-entry states, B the guard, and S the body transition.

B.1 Conjunction of Valid Invariants

Proof of Proposition 1. For each j, validity gives $P \Rightarrow I_j$; hence $P \Rightarrow \bigwedge_j I_j = I$. Suppose $I \wedge B$ holds before one execution of S. Then $I_j \wedge B$ holds for every j, so preservation of I_j establishes every I_j afterward. Therefore I is preserved and is a valid invariant.

Because a conjunction implies each conjunct, $I \preceq I_j$. It satisfies $I \prec I_j$ precisely when the reverse implication fails. Since

$$I_j \Rightarrow I \quad \Longleftrightarrow \quad I_j \Rightarrow \bigwedge_{\ell \neq j} I_\ell,$$

the stated condition follows. In particular, two valid invariants form a strict strengthening of both exactly when neither invariant implies the other; without this non-redundancy condition, conjunction is only guaranteed to be no weaker. □

B.2 Mutual Support Among Arbitrarily Many Clauses

Proof of Proposition 2. The first premise is exactly establishment of I_C. For preservation, assume $I_C \wedge B$ before executing S. The second premise establishes each c_j afterward. Since every conjunct then holds in the same successor state, their conjunction I_C also holds. Thus $\{I_C \wedge B\} S \{I_C\}$, and I_C is valid.

No standalone preservation premise appears in this argument. A clause may use all other conjuncts in the pre-state, so clauses that fail preservation separately may still be preserved jointly. Establishment is different: $P \Rightarrow I_C$ implies $P \Rightarrow c_j$ for every j. Hence a candidate false at some loop-entry state cannot be rescued by conjunction. □

The phenomenon is not limited to pairs. For any $m \geq 2$, consider the cyclic transition, where x_i' denotes the value of x_i after one execution of S,

$$x_1' = x_2, \quad x_2' = x_3, \quad \ldots, \quad x_{m-1}' = x_m, \quad x_m' = x_1$$

with entry condition $P \equiv \bigwedge_i x_i = 0$, and candidates $c_i \equiv x_i \geq 0$. Each c_i is established. It is not preserved alone: a state can satisfy $x_i \geq 0$ while its predecessor under the cyclic assignment is

negative. Nevertheless, the conjunction $\bigwedge_i x_i \geq 0$ is preserved because every successor coordinate is an old coordinate constrained by another conjunct. Thus all m candidates can fail standalone preservation while forming one valid, mutually supported invariant.

B.3 Greatest Jointly Inductive Candidate Subset

Let $C_0 = C$, and let $C_{t+1} \subseteq C_t$ be the set remaining after one ideal Houdini deletion round in Algorithm 1. Because C is finite, this descending sequence reaches the fixed point $\mathsf{H}_{\mathcal{L}}(C)$.

Proof of Theorem 3. At the fixed point, every $c \in \mathsf{H}_{\mathcal{L}}(C)$ satisfies establishment and preservation under the full conjunction $I_{\mathsf{H}_{\mathcal{L}}(C)}$. Applying Proposition 2 shows that this conjunction is valid.

It remains to prove greatestness. Let $C' \subseteq C$ be any jointly inductive subset. We show by induction over deletion rounds that $C' \subseteq C_t$. The base case $C' \subseteq C_0 = C$ is immediate. Assume $C' \subseteq C_t$, and take any $c \in C'$. Joint establishment of C' gives $P \Rightarrow c$, so c cannot be deleted for establishment. Joint preservation gives

$$\{I_{C'} \wedge B\}\, S\, \{c\}.$$

Since $C' \subseteq C_t$, the antecedent $I_{C_t} \wedge B$ implies $I_{C'} \wedge B$. Strengthening a valid Hoare precondition preserves validity, so

$$\{I_{C_t} \wedge B\}\, S\, \{c\}$$

also holds. Therefore no member of C' is deleted in round t, proving $C' \subseteq C_{t+1}$. At termination, $C' \subseteq \mathsf{H}_{\mathcal{L}}(C)$. Conjunction reverses set inclusion, hence $I_{\mathsf{H}_{\mathcal{L}}(C)} \preceq I_{C'}$. □

The theorem is deliberately candidate-relative: Houdini cannot synthesize a missing clause, so it need not recover the strongest invariant expressible in the full assertion language. It is also an ideal-oracle result. The inference-time Frama-C/WP backend may time out or fail to prove a valid obligation; such clauses are conservatively removed. The final retained conjunction remains accepted by the verifier, but completeness and greatestness are not claimed for these resource-bounded executions.

B.4 Closed Form of the Coverage-Game Shapley Value

The allocation of Equation (1) is the Shapley value of the coverage game $v(\mathcal{S}) = |\bigcup_{i\in\mathcal{S}} R_i|/|\mathcal{N}|$, for coalitions $\mathcal{S} \subseteq \mathcal{G}$ of the rollout group.

Proposition 4 (Closed form of the coverage game)**.** *For the coverage game $v(\mathcal{S}) = |\bigcup_{i\in\mathcal{S}} R_i|/|\mathcal{N}|$, the Shapley value is $\phi_i = \frac{1}{|\mathcal{N}|}\sum_{n\in R_i}\frac{1}{f(n)}$, and $\sum_i \phi_i = v(\mathcal{G})$.*

Proof. Fix a trace n rejected by $f(n)$ rollouts. In a uniformly random ordering of the group, n contributes to the marginal gain of exactly one player—the first of those $f(n)$ rollouts to appear—and by symmetry each of them is first with probability $1/f(n)$. Player i therefore receives expected credit $1/f(n)$ for each $n \in R_i$ and zero otherwise. Summing over $n \in \mathcal{N}$ and normalizing by $|\mathcal{N}|$ gives the closed form; efficiency follows because each covered trace distributes exactly one unit of credit among its $f(n)$ rejectors. □

C Implementation Details

Table 4 lists the implemented defaults. SFT synthesis, RL, and inference share one rollout–decompose–filter–compose implementation; RL applies Houdini once per response for credit assignment, whereas SFT and inference apply it after union (Appendix L).

The exact near-input tier order is `0, 1, 2, -1, 3, 5, 8, -2, 10, 17, 4, -3, 25, 6, 40, 7, -5, 13, 50, 9`.

Constant	Value	Role
runs requested	12	input executions; deterministic no-input programs collapse to one, and oracle-body sampling doubles the request
sampler seed	0	one canonical example set per reward request unless overridden
input tiers	20 fixed values	deterministic near-input schedule listed below; unconstrained tier candidates are clamped to $[-64, 64]$
far probes	3, plus 2 ordered multi-parameter probes	seed-hashed input-envelope coverage, magnitude ≤ 512
relation deltas	dense: $\pm\{1, 2\}$; terminal: $\pm\{1, 2, 3, 5, 8\}$	single-axis and pairwise perturbations; terminal bases require a witnessed final transition rather than a forward dense window
dense window	3	sampled successors required for a relation base
relation base cap	96	bases stratified across positives
post-exit steps	24	continuation length after a genuine exit (one negative trace per run)
negative-example budgets	48 / 12	relational / post-exit traces (sampler schema v7); stable round-robin across structural buckets, nearest perturbations first
iteration cap	2×10^6	divergence safety net (capped runs flagged)
(w_c, w_g)	$(1.0, 0.3)$	negative rejection / Shapley group-credit weights
w_d	0.02	cost per conservative duplicate; unique zero-coverage clauses are not penalized
K	20 clauses	response cap applied independently to every generated response
w_o	0.05	overflow cost per clause after K
Houdini iterations	≤ 500	greatest-inductive-subset pruning
WP configuration	Alt-Ergo, 30 s, Typed model	current implementation default and per-goal budget

Table 4: Defaults for execution sampling (§4.3), reward computation (§4.5), inference, and verification.

C.1 Negative-Trace Sampling

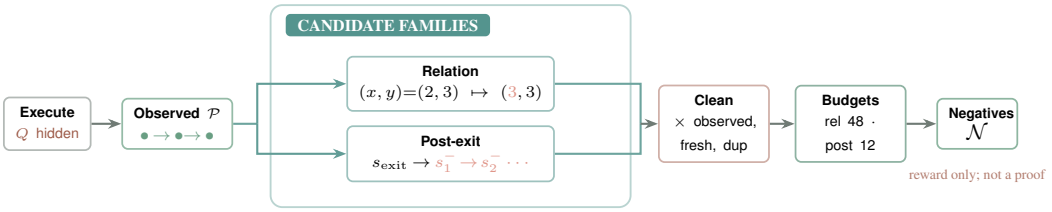

Figure 5: **Construction of target-hidden negative traces.** Concrete executions provide observed reachable states. Two transformations probe relational structure and behavior past a genuine exit. Conservative cleaning and per-family budgets produce $\mathcal{N}$, which is used only for RL reward computation and never exposes Q.

Algorithm 2 summarizes the sampler used for RL. Trace records real loop-head states and, after each genuine exit, continues the real loop body to obtain one post-exit trace. $\mathrm{Perturb}_\Theta$ applies the configured single-variable and pairwise changes; the exact perturbations and budgets are those in Table 4.

The two families are complementary and both are hard by construction. A relation perturbation is admitted only when it stays inside the sampled envelope of its context, preserves the guard truth value,

and does not lie on the locally witnessed transition tangent, so only clauses that constrain relations among variables can reject it. A post-exit continuation executes the real body past a witnessed exit into states that no perturbation of an observed state can produce, so only clauses that pin down the exit boundary reject it. Families whose witnesses are rejected by boilerplate bounds (single-variable range escapes and frame equalities) were removed after offline audits. Section 5.6 ablates the structured families against uniform random perturbations.

Algorithm 2 Target-hidden negative-trace sampling

Input: masked loop $\mathcal{L} = (P, B, S)$, input schedule U, configuration Θ
Output: reachable states $\mathcal{P}$, negative traces $\mathcal{N}$

$\mathcal{T}_{\text{exec}}, \mathcal{T}_{\text{post}}, c \leftarrow \text{Trace}(\mathcal{L}, U, \Theta)$ ▷ *c: some run was capped*
$\mathcal{P} \leftarrow \text{UniqueHeads}(\mathcal{T}_{\text{exec}})$; $M \leftarrow \text{SafeModifiedVars}(S)$
if $M = \varnothing$ or S has unsupported state **then return** $(\mathcal{P}, \varnothing)$
$\mathcal{P}_{\text{sample}} \leftarrow \text{StratifiedSample}(\mathcal{P})$
$C_{\text{rel}} \leftarrow \text{Perturb}_{\Theta}(\text{Dense}(\mathcal{P}_{\text{sample}}), M)$
if $\neg c$ **then** $C_{\text{rel}} \leftarrow C_{\text{rel}} \cup \text{Perturb}_{\Theta}(\text{Terminals}(\mathcal{T}_{\text{exec}}), M)$
$C_{\text{rel}} \leftarrow \text{TangentAndEnvelopeFilter}(C_{\text{rel}}, \mathcal{T}_{\text{exec}})$
$(C_{\text{rel}}, \mathcal{T}_{\text{post}}) \leftarrow \text{Clean}_{\mathcal{P}}(C_{\text{rel}}, \mathcal{T}_{\text{post}})$
$\widehat{C}_{\text{rel}}, \mathcal{N}_{\text{post}} \leftarrow \text{BudgetAndRoundRobin}(C_{\text{rel}}, \mathcal{T}_{\text{post}}, \Theta)$
$\mathcal{N} \leftarrow \{\langle s \rangle : s \in \widehat{C}_{\text{rel}}\} \cup \mathcal{N}_{\text{post}}$
return $(\mathcal{P}, \mathcal{N})$

$\text{Clean}_{\mathcal{P}}$ removes observed reachable states, possible fresh loop entries, duplicates, and unsupported candidates; a cleaned post-exit continuation remains one trace, and relation candidates become singleton traces. Within each structural bucket the round-robin prefers the smallest counterfactual change, so surviving witnesses sit close to the sampled manifold.

C.2 Group-Relative Optimization Details

For a sampled response group $A_{1:G} \sim \pi_{\theta_{\text{old}}}(\cdot \mid \mathcal{L})$, we normalize Equation 2 within its group,

$$\widehat{\text{Adv}}_i = \frac{r_i - \text{mean}_j\, r_j}{\text{std}_j\, r_j + 10^{-6}},$$

and use the clipped GRPO objective (Shao et al., 2024)

$$\mathcal{J}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_i \frac{1}{T_i}\sum_t \min\Big(\rho_{i,t}\widehat{\text{Adv}}_i, \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\widehat{\text{Adv}}_i\Big) - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})\right]. \quad (3)$$

Here T_i is the token length of response i, $\rho_{i,t}$ is the token-level ratio to the rollout policy, and π_{ref} is frozen. Duplicate detection normalizes comparison direction, equality sides, parentheses, immediate commutative operands, harmless identities, and Boolean association, but avoids transformations that require additional arithmetic assumptions. Unique zero-coverage clauses are not penalized because they may support another clause or the hidden target. Both coverage and Shapley credit lie in $[0, 1]$, so the unpenalized full reward lies in $[0, 1.3]$; duplicate and overflow costs can lower it. When the sampler retains no negatives, scoring falls back to binary inductiveness for a nonempty response.

D Generation Prompt

The canonical evaluation template appears below; every target and non-contract comment is stripped before insertion, and general annotation rules are supplied by `system_prompt.txt`. We rebuild both training sets with this single prompt pair. From archival pools of 37,958 RL and 3,454 SFT records, sanitization retains 37,481 and 3,200 records, removing records outside the scalar-integer single-loop interface, rows with C/ACSL parse or name-resolution errors, and answers with no surviving clause. SFT answers are parsed into clauses, conservatively deduplicated, stripped of malformed, out-of-scope, tautological, and dominated clauses, rewritten onto the released scalar interface, and checked by the same Frama-C/WP Houdini filter used by the method (5-second limit

per proof obligation), reducing 32,883 archived clauses to 23,913 verified, non-tautological clauses with at most 20 clauses per answer. No final RL or SFT model prompt contains a target clause or non-contract comment; the archival full source remains only in RL reward metadata, where the service strips its target before sampling and inductiveness filtering, and model-visible evaluation sources contain no provenance or outcome comments. The released training sets are further curated from this sanitized pool (Appendix G).

Generation prompt

```
You are given a C function. Produce the strongest inductive ACSL loop
invariants that capture the loop's behavior (conservation/relational laws
+ tight progress bounds). Output ONLY invariant lines, each formatted
exactly as:
loop invariant <expr>;
Output at most 20 invariant lines.

Program:
{program}
```

The accompanying system prompt appears verbatim below (also released as `prompt/system_prompt.txt`).

System prompt

```
You are a formal verification expert specializing in ACSL loop invariant
synthesis for Frama-C/WP.

## LOOP INVARIANT DEFINITION

A loop invariant is a property that:
1. Holds before the first loop iteration (initialization).
2. Is preserved by every loop iteration when the loop guard is true
   (inductiveness).
3. Combined with loop exit (`!guard`), helps establish required
   loop-exit facts.

## RULES

- Add one core conservation/relational equality that explains loop
  updates (state transition law).
- Add minimal progress bounds for loop counters and key arithmetic
  variables.
- Prefer strong invariants; avoid weak/tautological ones.
- Do not add unrelated invariants just to increase count.
- MUST NOT modify or rewrite any `unknown()`/`unknownN()` call.
- `\at(v, Pre)` can ONLY be used with function parameters, never with
  local variables initialized inside the function.
- `\at(v, LoopEntry)` denotes the value of v at the start of the loop
  and can ONLY be used with local variables initialized before the
  loop (using v = unknown();).
- Loop guard is NOT an invariant; weaken strict guards to non-strict
  bounds at exit.
- Parenthesize mixed equality and relational expressions so their
  grouping is explicit.
- Use `loop invariant` only as a line prefix, never inside expressions.
- Emit scalar invariant expressions only; do not declare predicates,
  logic functions, axioms, or lemmas.
- Do NOT use the ternary `? :` operator; split cases with `==>` instead.
- `^` is bitwise XOR in C/ACSL, not exponentiation --- write `x*x*x`
  for x cubed.
- Do NOT place a C boolean directly in arithmetic (`x + (a > b)` is
  invalid); use `==>` to split cases.
- Only use variables declared in the given function.
- Operators: comparison `==` `!=` `<` `<=` `>` `>=`; logical `&&` `||`
  `!` `==>` `<==>`; arithmetic `+` `-` `*` `/` `%`.

## ACSL INVARIANT SYNTAX

Allowed expression forms (all verified by Frama-C/WP):

1. **Bounds**: `0 <= i`, `i <= n`, `x >= 1`
2. **Linear equality**: `i + c == n`, `2 * s == i * (i - 1)`,
   `x == q * y + r`
3. **Polynomial equality**: `x == n * n * n`,
   `6 * s == i * (i + 1) * (2 * i + 1)`
```

```
4. **Relational identity** (multi-var algebraic law):
   `1 + x * z - x - z * y == 0`, `p * s - r * q == 1`
5. **Implication**: `i > 0 ==> s >= i`, `a != 0 ==> q >= 1`
6. **Modular**: `i % 5 == 0`, `r >= 0 && r < y`
7. **Bitshift** (power-of-2 only): `p == 1 << i`, `s == (1 << i) - 1`
8. **Conjunction**: `c >= 1 && c <= k`
9. **\at for params**: `n == \at(n, Pre)` (function parameters only)

Forbidden:
- `^` for exponentiation (it is XOR) --- use repeated `*` or `<<` for
  powers of 2.
- `\product`, `\sum`, `\factorial` --- not ACSL scalar operators.
- `? :` ternary --- rewrite with `==>`.
- `\old(x)` --- use `\at(x, Pre)`.
- Type casts `(int)`, `(long)` --- unnecessary in ACSL logic.
- Function calls, predicates, lemmas, axioms --- emit only scalar
  expressions.

Operators (complete list):
  comparison: `==` `!=` `<` `<=` `>` `>=`
  logical:    `&&` `||` `!` `==>` `<==>`
  arithmetic: `+` `-` `*` `/` `%` `<<` `>>`

## OUTPUT

Return ONLY invariant lines, one per line, formatted exactly as:
`loop invariant <property>;`
Do not return code fences, `loop assigns`, the surrounding C function,
or explanations.
```

E TRAINING INTERFACE

The training service is packaged as a self-contained, CPU-only container with gcc, the verifier, Why3, and its prover backend. It exposes `POST /reward`, accepting program source, an array of model responses, `reward_variant` in {`binary`, `whole_coverage`, `base`, `base_shapley`, `full`}, and `sampler.negative_sampler` in {`random`, `structured`}; the two switches are independent and default to `full` and `structured`. The response reports each rollout's reward together with its coverage, Shapley-credit, redundancy, and overflow components. Responses may be raw LLM text, invariant lists, or annotated code; an offline scorer provides the same generation reward over JSONL/Parquet rollout dumps.

F Training–Evaluation Overlap Audit

We compare the released RL (5,000 records) and SFT (14,858 records) training sets against all 832 evaluation programs after target removal, under three increasingly aggressive normalizations: exact text, alpha-renamed identifiers, and alpha-renaming with integer constants abstracted. No training record matches any evaluation program at any level; this is enforced during curation and re-checked on the released files (Table 5). Structural similarity is expected for a scalar single-loop language and is quantified rather than denied: the total-variation distance between the training and evaluation distributions over structural cells (control-flow profile, guard kind, nonlinearity, nondeterminism, variable band) is 0.42 for RL and 0.63 for SFT, and 702 of 832 evaluation programs share a cell with at least one training program. This audit covers the released fine-tuning data, not base-model pretraining corpora.

	RL (5,000)	SFT (14,858)
Exact-text matches	0	0
Alpha-renamed matches	0	0
Alpha-renamed, constant-abstracted matches	0	0
TV distance over structural cells	0.42	0.63

Table 5: Overlap audit of the released training sets against the 832 target-removed evaluation programs.

G Training-Pool Curation

The released training sets are filtered and synthesized from the 37,481-record sanitized pool of Appendix D: programs with no training signal and too-easy or too-hard programs are removed, the selection is aligned with the evaluation distribution over structural cells (control-flow profile, guard kind, nonlinearity, nondeterminism, variable band), and SFT targets are minimal inductive invariant sets synthesized with `gpt-5.6-luna` and verified by Frama-C/WP. Per-stage counts are emitted as machine-readable reports shipped with the code. Table 6 summarizes the released sets.

RL set	5,000 records over 1,131 distinct loop shapes
SFT set	14,858 records (3,058 archival + 11,800 synthesized)
Synthesized answers	mean 5.07 clauses (median 4); 98.8% contain a relational transition law
RL–SFT overlap	180 programs

Table 6: Released training sets.

The reward discriminates on the released data. On the 125 released RL programs with a reference answer, the production reward gives the reference a median of 0.845 and its bounds-only projection a median of 0.0; the tautology `1 == 1`, the loop guard copied as an invariant, and an unsound clause score exactly zero on every program. A simulated eight-answer group has a median within-group reward standard deviation of 0.21, so group-relative updates retain a usable advantage signal.

H Experimental Protocol Details

The released sets (Appendix G) contain 5,000 RL and 14,858 SFT records with 180 programs in common. The 832-file evaluation workload contains 316 linear programs (133 from Code2Inv, 84 from SyGuS 2019, and 99 from SV-COMP 2024), 50 nonlinear programs (30 from LIPuS and 20 from SV-COMP 2024), and the 466 integer programs in Loopy's original 469-program corpus. Three floating-point programs are excluded, and unsupported inputs remain failures in the common denominator.

Before model invocation, we remove `assert` and `ensures` predicates, executable assertion calls, conventional error-label names, and non-contract comments, including source-provenance paths;

preconditions and executable semantics remain visible. CRAFT evaluation runs use the canonical generation template, extractor, semantic deduplication, and Houdini implementation; external tools retain their own orchestration. Each model retains its serving interface and chat template. In CRAFT, API calls set `reasoning_effort=none`; all locally served checkpoints disable thinking. We set temperature 1.0 and top-p 1.0, without an additional top-k, repetition penalty, re-roll, or target feedback. The cap is 8,192 completion tokens for API models, including any provider-internal reasoning tokens, and 2,048 emitted tokens for local vLLM models. Each RQ1 curve reuses the same stochastic 100-response batch per task; no per-response sampling seed is fixed.

Training configuration. Table 7 separates settings recoverable from the released pipeline from fields that still require the original trainer manifest. The final submission must replace every TBD; reward and sampler constants are already reported in Table 4.

Field	Value	Scope or required provenance
Backbone ID and revision	TBD	exact Qwen3-8B repository and immutable revision
SFT optimizer / schedule	TBD	optimizer, learning rate, warmup, decay, and precision
SFT batch / duration	TBD	global batch, accumulation, epochs or updates, and sequence length
RL algorithm	GRPO	group-normalized objective in Equation 3
RL optimizer / schedule	TBD	optimizer, learning rate, warmup, decay, and precision
Rollout group size G	TBD	responses sharing one group-relative update
RL batch / duration	TBD	prompt batch, accumulation, effective batch, and update count
Clipping / KL	TBD	ε, β, and reference-policy update convention
Training seeds	TBD	seed for each reported checkpoint and ablation run
Hardware / software	TBD	GPU type and count, trainer, vLLM, CUDA, and framework versions
Local rollout decoding	non-thinking; $T = 1$, top-$p = 1$; 2,048 tokens	data synthesis, RL, and local evaluation

Table 7: Training configuration and required run-manifest fields.

I Complete Experimental Data

This section collects the per-model, per-stratum, and per-seed measurements behind the main aggregates. No entry marked TBD is used in the analysis.

I.1 PROBE CURVES

Table 8: Complete probe results. k_{95} is the smallest measured $k \in \{1, 4, 8, 16, 32\}$ at which compose@k reaches 95% of its value at $k = 32$.

Whole-response pass rates

Model	pass@1	pass@4	pass@8	pass@16	pass@32
Qwen3-0.6B	0.58%	TBD	TBD	TBD	TBD
Qwen3-4B	3.49%	TBD	TBD	TBD	TBD
Qwen3-8B	3.55%	TBD	TBD	TBD	TBD
Qwen3-14B	4.99%	TBD	TBD	TBD	TBD
Qwen3-30B-A3B	5.00%	TBD	TBD	TBD	TBD
Llama 3.1 8B	0.48%	TBD	TBD	TBD	TBD

Clause-composition rates

Model	compose@1	compose@4	compose@8	compose@16	compose@32	k_{95}
Qwen3-0.6B	9.50%	TBD	TBD	TBD	TBD	TBD
Qwen3-4B	23.08%	TBD	TBD	TBD	TBD	TBD
Qwen3-8B	26.32%	TBD	TBD	TBD	TBD	TBD
Qwen3-14B	30.17%	TBD	TBD	TBD	TBD	TBD
Qwen3-30B-A3B	28.73%	TBD	TBD	TBD	TBD	TBD
Llama 3.1 8B	18.03%	TBD	TBD	TBD	TBD	TBD

Table 9: Probe results by benchmark stratum (316 linear, 50 NLA, 466 Loopy, 832 overall programs) at k=1 and k=8, laid out as in Table 16: pass@k (pass) and compose@k (comp) percentages side by side. Best value per column within each budget block in bold.

	Linear		NLA		Loopy		All	
k	pass	comp	pass	comp	pass	comp	pass	comp
Qwen3-0.6B								
1	0.55	6.96	0.02	2.00	0.66	12.02	0.58	9.50
8	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B								
1	2.80	23.42	0.00	2.00	4.34	25.11	3.49	23.08
8	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-8B								
1	2.79	25.63	0.00	2.00	4.44	29.40	3.55	26.32
8	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-14B								
1	4.36	**31.01**	**0.12**	0.00	**5.94**	**32.83**	4.99	**30.17**
8	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-30B-A3B								
1	**5.22**	28.80	0.02	0.00	5.38	31.76	**5.00**	28.73
8	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Llama 3.1 8B								
1	0.38	16.77	0.00	**4.00**	0.60	20.39	0.48	18.03
8	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 10: Per-GPU decoding throughput of the base probe checkpoints, measured with vLLM on A800-80G GPUs (n=100 responses per task, temperature 1.0, `max_tokens=2048`).

Model	tok/s/GPU
Qwen3-0.6B	18,727
Qwen3-4B	7,445
Qwen3-8B	5,833
Qwen3-14B	3,168
Qwen3-30B-A3B	5,031
Llama 3.1 8B	5,135

I.2 SFT Probe Curves

The SFT checkpoints (one epoch on the final SFT corpus) are evaluated under the same 832-task protocol as the base probe: 100 saved target-hidden responses per task at temperature 1.0, 5-second per-obligation prover budget, and the current prompt.

Table 11: Complete SFT probe results. k_{95} is the smallest measured $k \in \{1, 4, 8, 16, 32\}$ at which compose@k reaches 95% of its value at $k = 32$.

Whole-response pass rates

Model	pass@1	pass@4	pass@8	pass@16	pass@32
Qwen3-4B + SFT	7.40%	TBD	TBD	TBD	TBD
Qwen3-8B + SFT	7.10%	TBD	TBD	TBD	TBD
Qwen3-14B + SFT	8.26%	TBD	TBD	TBD	TBD
Llama 3.1 8B + SFT	5.12%	TBD	TBD	TBD	TBD

Clause-composition rates

Model	compose@1	compose@4	compose@8	compose@16	compose@32	k_{95}
Qwen3-4B + SFT	26.08%	TBD	TBD	TBD	TBD	TBD
Qwen3-8B + SFT	25.60%	TBD	TBD	TBD	TBD	TBD
Qwen3-14B + SFT	29.33%	TBD	TBD	TBD	TBD	TBD
Llama 3.1 8B + SFT	24.04%	TBD	TBD	TBD	TBD	TBD

Table 12: Per-GPU decoding throughput of the SFT checkpoints, measured with vLLM on A800-80G GPUs (n=100 responses per task, temperature 1.0, `max_tokens=2048`).

Model	tok/s/GPU
Qwen3-4B + SFT	7,445
Qwen3-8B + SFT	5,833
Qwen3-14B + SFT	3,168
Llama 3.1 8B + SFT	5,135

I.3 Reinforcement-Learning Results

Table 13: Complete comparison before and after RL. The Zero and SFT initializations are evaluated under the same decoding and verification configuration.

Path	Model	pass@1	compose@1	compose@4	compose@16	compose@32	k_{95}
Zero	Base 8B	3.55%	26.32%	TBD	TBD	TBD	TBD
Zero	Base 8B + RL	8.77%	53.85%	63.82%	70.19%	71.51%	16
SFT	SFT 8B	7.10%	25.60%	TBD	TBD	TBD	TBD
SFT	SFT 8B + RL	TBD	TBD	TBD	TBD	TBD	TBD

I.4 Ablation Results

The reward variants correspond directly to the two levels of Equation 2. *Binary Inductiveness* assigns one iff the capped, normalized response is nonempty and every clause survives Houdini. *Whole-Rollout Strength* uses negative coverage only when every capped clause survives,

$$r_i^{\text{whole}} = \mathbf{1}[C_i^{\text{train}} = \text{set}(\text{dedup}(\bar{A}_i))] \frac{|R_i|}{|\mathcal{N}|} - w_o o(A_i).$$

Clause-Decomposed Strength removes this all-or-nothing indicator and scores the surviving C_i^{train}; the *Full Compositional Credit* additionally adds $w_g \phi_i$ and subtracts $w_d d_{\text{dup}}(A_i)$. These are `binary`, `whole_coverage`, `base`, and `full`, respectively. All variants train from the Zero initialization on the curated pool with identical optimization and inference settings.

Table 14: Reward-function ablation on Qwen3-8B, trained from the Zero initialization (%); exact numbers for Figure 4. Best result per column among trained variants in bold.

	pass@k				compose@k			
Variant	1	4	16	32	1	4	16	32
Untrained 8B (reference)	3.55	TBD	TBD	TBD	26.32	TBD	TBD	TBD
Binary Inductiveness	9.88	13.33	17.08	19.30	8.77	15.62	24.04	27.88
Whole-Rollout Strength	**25.35**	**28.10**	**30.10**	**31.32**	27.76	29.21	31.13	33.17
Clause-Decomposed Strength	4.60	10.16	16.27	19.03	**58.29**	**65.99**	69.35	70.19
Full Compositional Credit (ours)	8.77	15.25	20.69	22.92	53.85	63.82	**70.19**	**71.51**

The negative-trace sampler comparison uses matched Qwen3-8B runs from the Zero initialization that differ only in the sampler. Random uses uniform local perturbations, whereas Structured uses the relation and post-exit families with matched budgets.

	pass@k				compose@k			
Negative sampler	1	4	16	32	1	4	16	32
Random	8.63	14.40	19.43	21.82	52.04	60.46	67.31	68.75
Structured (ours)	**8.77**	**15.25**	**20.69**	**22.92**	**53.85**	**63.82**	**70.19**	**71.51**

Table 15: Negative-sampler ablation (%). Bold marks the better sampler for each metric.

Structured sampling improves compose@1 and compose@32 by 1.8 and 2.8 points and is at least as good at every budget under both metrics.

I.5 Cross-Model Main Results

Table 16: Cross-model results by benchmark stratum (316 linear, 50 NLA, 466 Loopy, 832 overall programs). Each stratum reports pass@k (pass) and compose@k (comp) percentages side by side at the rollout budget k in the first column. reasoning_effort=none throughout. For every model, all rows come from one archived rollout pool per task (ten rollouts; eight for GPT-5.6-luna): compose@k unions the first k rollouts through the inference filter chain, and pass@k uses the unbiased estimator over per-rollout verdicts. GPT-5-mini has no reasoning-off setting and is therefore not comparable here.

	Verified (%)							
	Linear		NLA		Loopy		All	
k	pass	comp	pass	comp	pass	comp	pass	comp
GPT-5-nano								
1	3.77	34.81	0.40	22.00	3.80	34.55	3.58	33.89
4	10.79	48.10	1.60	30.00	11.48	51.72	10.62	49.04
8	15.84	55.38	3.20	44.00	17.58	56.87	16.05	55.53
GPT-5								
1	35.06	52.22	5.80	36.00	33.78	52.36	32.58	51.32
4	47.73	64.87	8.74	52.00	50.67	67.38	47.03	65.50
8	51.93	70.25	9.60	66.00	56.13	73.82	51.74	72.00
GPT-5.6-luna								
1	81.05	84.18	73.00	82.00	82.40	86.05	81.32	85.10
4	89.09	89.56	85.94	90.00	90.12	91.85	89.48	90.87
8	89.87	90.19	90.00	90.00	91.63	93.13	90.87	91.83
Claude Sonnet-4.6								
1	34.49	55.38	7.80	44.00	40.67	59.44	36.35	56.97
4	40.32	62.97	8.80	52.00	47.27	66.31	42.32	64.18
8	42.87	66.46	9.60	64.00	50.12	67.60	44.93	66.95
DeepSeek V4-Flash								
1	28.67	52.22	5.00	44.00	30.15	51.72	28.08	51.44
4	42.35	66.46	7.52	60.00	47.73	69.96	43.27	68.03
8	46.64	71.52	8.00	72.00	53.59	76.61	48.21	74.40

Table 17: Reasoning-on reference rows complementing Table 16: *default* leaves reasoning_effort unset (the provider's own default); *medium* sets it explicitly.

Model	Reasoning	k	Linear	NLA	Loopy	All
DeepSeek-V4-Flash	default	1	57.91/62.03	4.00/8.00	52.15/54.51	51.44/54.57
GPT-5-mini	default	1	55.70/77.85	4.00/10.00	60.94/75.75	55.53/72.60
GPT-5	medium	1	77.85/80.38	14.00/14.00	80.47/80.47	75.48/76.44

All rows are full-workload means over 832 programs. Each pass@k/compose@k pair at a given k is certified from the same saved k responses.

I.6 Comparison with Specialized Tools

Table 18: Archived target-hidden tool comparison: the `reasoning_effort=none` rows from the main text alongside the archived `reasoning_effort=medium` rerun; Daikon makes no model call and is listed once. Each cell reports the verification rate under the common file-level denominator.

Method	Reasoning	Linear / 316	NLA / 50	Loopy / 466	Overall / 832	Tokens / task	Time / task
AutoSpec	none	47.78%	6.00%	42.27%	42.19%	2,181.72	27.34 s
AutoSpec	medium	65.82%	8.00%	64.81%	61.78%	25,154.54	54.77 s
SESpec	none	28.80%	4.00%	33.05%	29.69%	22,081.61	68.50 s
SESpec	medium	56.96%	6.00%	49.57%	49.76%	44,807.05	117.31 s
Clause2Inv	none	20.89%	0.00%	0.00%	7.93%	1,056.25	8.41 s
Clause2Inv	medium	19.94%	2.00%	0.00%	7.69%	385.80	29.04 s
Daikon	—	23.10%	0.00%	21.67%	20.91%	0	4.89 s
Loopy	none	20.25%	0.00%	19.74%	18.75%	14,513.37	147.16 s
Loopy	medium	72.78%	12.00%	70.82%	68.03%	111,973.15	381.01 s
Naive	none	15.19%	2.00%	18.88%	16.47%	225.95	2.98 s
Naive	medium	48.73%	8.00%	47.42%	45.55%	4,493.37	28.87 s
CRAFT (pass@1)	none	2.53%	0.00%	4.72%	3.61%	1,135.83	7.86 s
CRAFT (pass@1)	medium	61.08%	14.00%	54.08%	54.33%	6,928.30	53.52 s
CRAFT (compose@1)	none	49.68%	8.00%	45.71%	44.95%	1,135.83	29.66 s
CRAFT (compose@1)	medium	64.24%	14.00%	63.09%	60.58%	6,928.30	66.60 s
CRAFT (compose@4)	none	TBD	TBD	TBD	TBD	TBD	TBD
CRAFT (compose@4)	medium	TBD	TBD	TBD	TBD	TBD	TBD
CRAFT (compose@8)	none	TBD	TBD	TBD	TBD	TBD	TBD
CRAFT (compose@8)	medium	TBD	TBD	TBD	TBD	TBD	TBD

The main text reports each method's `reasoning_effort=none` row. The shared model-call parameters for the non-reasoning comparison are summarized separately in Table 19.

Table 19: Sampling parameters used in the non-reasoning target-hidden tool comparison.

Method	Sampling budget / task	Temperature / top-p	Completion cap	Reasoning
AutoSpec	8	1.0 / 1.0	8,192	none
SESpec	1 initial + up to 3 refinements per phase	1.0 / 1.0	8,192	none
Clause2Inv	adaptive; 1 per call	1.0 / 1.0	8,192	none
Daikon	no model call	–	–	–
Naive	1	1.0 / 1.0	8,192	none
CRAFT	1, 5, or 2×5	1.0 / 1.0	8,192	none
Loopy	15 initial + up to 7 repairs	0.7 / 1.0	8,192	none

Token totals are means per task: AutoSpec, SESpec, and Clause2Inv report them over 828, 826, and 821 tasks while their accuracy retains the 832-task denominator, and the 466 unsupported Clause2Inv tasks make no model call. The retained CRAFT pass@1 timing covers the 466 Loopy tasks; pass@1 and compose@1 reuse the same first model response.

J Detailed Limitations

Program scope. Our implementation targets numerical programs with one scalar-integer loop. This scope follows the dominant setting in prior loop-invariant benchmarks, but excludes nested loops and invariants over arrays, heaps, or recursively defined predicates. Such invariants require richer representations and proof dependencies, while available training corpora contain too few examples to support the discriminative signal used here. Our state perturbations cannot yet construct reliably informative negatives for inductive predicates or interacting nested loops (Liu et al., 2024a).

Inference engine. In preliminary comparisons, locally evaluated checkpoints achieved lower verification rates under vLLM than under alternative backends despite nominally matched decoding settings. We nevertheless use vLLM for all local base and trained checkpoints because its throughput makes broad sampling practical and its consistent use avoids an additional train–test mismatch. API models retain their provider serving interfaces and chat templates under the explicitly reported decoding and reasoning controls. Local-model success rates may therefore be conservative rather than engine-independent ceilings.

Verifier and specification language. Our filtering and all final judgments use Frama-C/WP, and candidates use ACSL. Parser behavior, generated obligations, prover integration, and syntax affect which clauses survive. The pipeline is not intrinsically tied to this toolchain, but the quantitative conclusions may not transfer unchanged to other verifiers, specification languages, or source languages.

Finite candidate-pool approximation. Before Houdini, the pipeline implementation applies a lightweight AST-based filter to parse, normalize, and deduplicate candidate clauses. This filter is deliberately conservative and does not provide a completeness guarantee for the full ACSL expression language: a semantically useful clause that is not handled by its restricted parser can be omitted. In addition, compose@k caps the candidate union passed to verification at 300 clauses to keep WP cost bounded. When an extreme response pool exceeds this limit, later clauses may be truncated even if they add useful information. Both effects can make the reported compose@k underestimate an ideal unbounded implementation. They affect candidate recall, not the validity of reported successes, because every retained result must still pass the full Frama-C/WP final judge.

Feedback regime. We do not evaluate iterative optimization from verifier feedback. In pilot runs, models quickly reached valid but weak invariants: they passed inductiveness checks, but with Q hidden the verifier did not identify the missing behavioral strength. Refinement therefore yielded little marginal gain, which decreased further as larger initial rollout unions captured the easy corrections. Our results characterize target-independent training and finite-budget composition, not settings with rich counterexamples or target-directed repair.

K Why Thinking Is Disabled for Local Models

For the non-reasoning API experiments reported in the main paper, GPT-family requests set `reasoning_effort=none`, compatible endpoints disable thinking explicitly, and Claude Sonnet disables adaptive thinking with a zero thinking budget. For all locally served checkpoints, including the base probes and models trained in our pipeline, we disable chain-of-thought (CoT) during synthetic data construction, training rollouts, and evaluation. Applying the same setting at all three stages avoids a train–test distribution shift. This choice has two practical motivations.

CoT substantially increases generation cost. Our inference procedure samples many rollouts for each program. Explicit CoT adds reasoning tokens and generation latency to every rollout, so its cost is multiplied by the sampling budget. The required output, however, is only a short, machine-parseable set of ACSL clauses. We therefore omit explicit CoT to keep both data generation and multi-rollout inference efficient.

Invariant clauses compose, but CoTs do not. A single rollout may contain both valid and invalid candidate invariants. The invariant output is explicitly decomposable into clauses: Houdini can

remove a failing clause while retaining the useful clauses, and the retained clauses can then be composed across rollouts. A CoT does not have this property. Correct and incorrect reasoning may be interleaved in one trace, with no reliable clause-level correspondence that would let us remove the reasoning associated with a rejected clause. Nor is there a principled way to merge the remaining fragments from different rollouts into one faithful CoT for the composed invariant set. Generating a new rationale after filtering would only produce a post-hoc explanation. Consequently, CoT supervision is not aligned with the clause-wise filter-and-compose target. For consistency, CoT remains disabled throughout the local-model pipeline.

L Training–Inference Mismatch in Houdini Ordering

Let $\bar{A}_i$ be the normalized, capped clause sequence proposed by rollout i, and let $C_i^{\mathrm{pf}} = \mathsf{PF}_{\mathcal{P}}(\mathrm{set}(\bar{A}_i))$. Recall that $\mathsf{Filter}_{\mathcal{L}}(C)$ denotes the implemented filtering pipeline on candidate set C. SFT construction and inference first merge the candidates and then filter them:

$$C^{\mathrm{infer}} = \mathsf{Filter}_{\mathcal{L}}\left(\bigcup_{i=1}^{G} \mathrm{set}(\bar{A}_i)\right). \tag{4}$$

This order allows a clause from one response to be preserved with the help of a companion clause proposed by another response.

RL requires a scalar reward for each rollout. The current implementation therefore runs Houdini independently and computes coverage, Shapley credit, and redundancy from

$$C_i^{\mathrm{train}} = \mathsf{Filter}_{\mathcal{L}}(C_i^{\mathrm{pf}}). \tag{5}$$

It also evaluates $\mathsf{Filter}_{\mathcal{L}}(\bigcup_i C_i^{\mathrm{pf}})$ as a group-level diagnostic, but this shared result does not determine each rollout's token-level advantage.

Why the two orders are not equivalent. Houdini is not distributive over union:

$$\mathsf{Filter}_{\mathcal{L}}\left(\bigcup_i C_i^{\mathrm{pf}}\right) \neq \bigcup_i \mathsf{Filter}_{\mathcal{L}}(C_i^{\mathrm{pf}}) \quad \text{in general.} \tag{6}$$

For example, a clause from one rollout may fail preservation in isolation but become inductive when conjoined with a supporting clause from another rollout. Thus, two clauses rejected by separate Houdini runs can belong to an inductive conjunction after union. Because the training reward filters each response first, it misses the positive credit associated with this cross-rollout support.

Why the practical effect is small. This is missed positive credit, not a dedicated penalty for non-inductiveness. The reward assigns coverage and Shapley credit to surviving clauses and penalizes duplicates and overflow. A unique clause that fails standalone Houdini incurs no additional penalty, although the absence of coverage can still give its rollout a lower group-relative advantage. At inference time, raw candidates from all responses are unioned before Houdini, so a complementary pair remains available and can be retained jointly. The mismatch therefore leaves inference-time cross-response support intact but may under-train clauses that depend on it. Pilot audits suggest such rescues are rare, though we did not measure a formal rescue rate.

Why retain this approximation. Independent filtering gives a stable, parallelizable reward for every response. Assigning the same union-level score to all G responses is uninformative under group-relative normalization because the shared component cancels. Exact leave-one-out credit under a set-level value V,

$$\Delta_i = V\left(\mathsf{Filter}_{\mathcal{L}}\left(\bigcup_j C_j^{\mathrm{pf}}\right)\right) - V\left(\mathsf{Filter}_{\mathcal{L}}\left(\bigcup_{j\neq i} C_j^{\mathrm{pf}}\right)\right),$$

requires at least $G+1$ union-level Houdini evaluations per prompt and can still miss higher-order interactions. Shapley-style credit captures such interactions more faithfully but requires many subset evaluations. This would be the Shapley value of the union-and-Houdini verifier game, unlike the closed-form Shapley allocation of negative-set coverage used by our reward.

Implications. The ordering difference makes standalone Houdini a slightly imperfect reward surrogate for compose@k, but it does not alter inference-time union-and-Houdini; we retain the cheaper approximation and leave leave-one-out or subset-sampled credit to future work.